

1. Frontend: Web-приложение (Demo)

Для веб-версии, которая должна ощущаться как нативное приложение.

- Язык: TypeScript (строгая типизация необходима для сложных состояний чатов).
- Фреймворк: Next.js или Vite + React. Vite предпочтительнее для Demo из-за мгновенной сборки.
- State Management (Сердце мессенджера): Zustand или Nano Stores. Они максимально легкие и быстрые, в отличие от тяжелого Redux.
- Анимации:
 - Framer Motion: Для сложных переходов между экранами и плавного появления элементов.
 - GSAP: Для высокопроизводительных микро-взаимодействий.
 - Lottie / Rive: Для интерактивных стикеров и анимированных иконок (Rive лучше, так как позволяет управлять анимацией программно).
- Стилизация: Tailwind CSS (минимальный размер CSS-бандла).

2. Mobile & Desktop (Полноценные приложения)

Чтобы достичь «инженерных принципов Telegram», нужно выбирать инструменты с прямым доступом к GPU.

- Основной стек: Flutter (Dart).
 - Почему: Это лучший выбор для мессенджера с упором на анимации. Flutter отрисовывает интерфейс на собственном движке (Impeller/Skia), что дает идентичную и плавную картинку (120 Гц) на iOS, Android и Desktop.
- Альтернатива (Native): Swift (iOS) и Kotlin (Android). Если нужна абсолютно экстремальная производительность и минимальный размер APK.
- Desktop: Tauri (на базе Rust). Он гораздо легче и быстрее, чем Electron. Использует нативный WebView системы и Rust для логики.

3. Backend & Real-time (Инженерный слой)

Здесь реализуется то самое «бинарное соединение».

- База данных и Auth: Supabase (PostgreSQL).
 - Используем GoTrue для регистрации.
 - Используем PostgREST для обычных запросов.
- Real-time сообщения: Supabase Realtime (на базе Elixir/Phoenix).
 - Важно: Для реализации именно бинарного протокола (как в описании), поверх Supabase стоит поставить небольшой микросервис на Go (Golang) или Rust.
- Протокол передачи данных: Protocol Buffers (Protobuf) от Google.
 - Вместо тяжелого JSON данные передаются в сжатом бинарном виде. Это дает ту самую скорость < 200 мс.
- Кэширование: Redis. Для мгновенного хранения статусов «онлайн» и временных метаданных.

4. Работа с медиа и файлами

- Хранилище: Supabase Storage (S3-совместимое).
- Обработка изображений: Sharp (Node.js) или нативные библиотеки Go для создания превью (BlurHash), чтобы пользователь видел размытый фон до загрузки фото.

Итоговая схема стека Beam:

УровеньТехнологияПочему это в Beam?Язык логикиTypeScript / Dart / GoБезопасность типов и высокая скорость исполнения.Бинарный обменProtobuf + WebSocketsМинимум трафика, мгновенный отклик.UI / АнимацииFramer Motion + RiveЭффект «дорогого» и плавного продукта.Backend-as-a-ServiceSupabaseБыстрый старт, надежный Auth и готовая Postgres.ПроизводительностьWASM (WebAssembly)Для тяжелых вычислений (шифрование) в браузере.Локальная БДSQLite (через Drift в Flutter)Для мгновенного поиска по истории без интернета.